

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота №2 (новий зразок)

з дисципліни

«Компоненти програмної інженерії».

Частина 3. Архітектура програмного забезпечення»

Шарова архітектура та доменна модель

Виконали:

Александров Ігор Анатолійович

Балалаєв Максим Юрійович

Зінченко Дар'я Максимівна

Каленченко Михайло Олександрович

Перевірив:

Грибенко Єгор Володимирович

Київ 2026

Тема роботи: Шарова архітектура та доменна модель

Мета роботи: Відділити бізнес-логіку від інфраструктури. Студенти рефакторять код із лабораторної 1, щоб на практиці побачити різницю між «все в купі» та структурованим підходом.

Обрана предметна область: CRUD-бекенд застосунок для спортивного клубу. Керування тренуваннями, персоналом, клієнтами та планами

В ЛР-2 (старий зразок) було розроблено монолітний CRUD-застосунок із класичною шаровою архітектурою на базі ASP.NET та PostgreSQL та з використанням Dependency Injection. Використовувалася Anemic Domain Model

В ЛР-2 (новий зразок) було реструктуровано проект та виконано перехід від Anemic до Rich Domain Model.

Що змінилося в структурі проєкту порівняно з лабораторною 1?

Головна зміна полягає у відмові від монолітного підходу з анемічною моделлю (Anemic Domain Model) на користь строгого розділення відповідальності між шарами. Якщо раніше сутності, ймовірно, містили лише набори публічних властивостей (get/set), а вся логіка була «розмазана» по сервісах або контролерах, то тепер проєкт має чітку модульну структуру:

- **Domain (GymManagement.Domain):** став незалежним ядром системи. Введено базові класи AggregateRoot та Entity. Замість примітивних типів використовуються об'єкти-значення (ValueObjects - Email, Money, PhoneNumber, TimeRange). Сутності більше не мають публічних сеттерів, зміна стану відбувається через інкапсульовані методи.
- **Application (GymManagement.Application):** містить Use Cases системи. Тут розміщені сервіси (ClientService, MembershipService) та DTO (CreateClientDto, InvoiceDtos). Цей шар лише координує виконання завдань, але не містить логіки самої предметної області.
- **Infrastructure (GymManagement.Infrastructure):** відповідає за технічні деталі реалізації - роботу з базою даних (GymManagementContext), репозиторії, міграції та безпеку (JWT токени).
- **Presentation (Controllers):** максимально тонкий шар, який відповідає лише за прийом HTTP-запитів та повернення відповідей.

Які переваги дало розділення на шари?

- **Ізоляція бізнес-логіки (Separation of Concerns):** правила предметної області більше не залежать від бази даних, ORM чи веб-фреймворку. Помилки бази даних не впливають на логіку розрахунку абонементів.
- **Висока тестованість:** завдяки інверсії залежностей (використанню абстракцій-портів, таких як `IClientRepository` чи `IUnitOfWork`) стало можливо писати швидкі Unit-тести без підключення до реальної БД. Наявність папки `Tests/Unit` (наприклад, `ClientTests.cs`, `MembershipServiceTests.cs`) доводить, що доменну логіку тепер легко покрити тестами в ізоляції.
- **Захист цілісності даних:** дані не можуть бути змінені в обхід бізнес-правил, оскільки логіка створення об'єктів контролюється фабриками (наприклад, `ClassFactory`, `EnrollmentFactory`).

Які недоліки / ускладнення з'явилися?

- **Збільшення обсягу коду (Boilerplate):** навіть для створення простої CRUD-операції тепер потрібно задіяти багато файлів: `Controller`, `Service`, `DTO`, `Repository interface`, `Repository implementation` та `Domain Entity`.
- **Складність мапінгу (Mapping):** оскільки шари ізольовані, сутності предметної області не можуть напряду відправлятися клієнту. Виникла необхідність постійно перетворювати об'єкти (наприклад, перекладати дані з `Client` у `ClientDto`). Це вимагає додаткових зусиль та уважності до деталей.
- **Складніша навігація та крива навчання:** відстежити потік виконання коду стало складніше. Розробнику доводиться "стрибати" між шарами через абстракції, замість того, щоб побачити всю логіку в одному методі контролера.

Наскільки простіше тепер змінити БД або фреймворк?

Архітектура зробила цей процес максимально безболісним. Оскільки шар `Domain` та `Application` залежать виключно від абстрактних портів (папка `Ports` в `Domain`-шарі), система нічого не знає про `Entity Framework Core` чи `PostgreSQL`. Щоб змінити базу даних (наприклад, перейти на `MongoDB` або використати мікро-ORM `Dapper`), достатньо написати нові реалізації репозиторіїв та контексту в шарі `Infrastructure`. Жоден рядок коду в бізнес-логіці (`Domain`), сервісах (`Application`) або тестах при цьому переписувати не доведеться.

Чому було обрано Rich Domain Model (а не Anemic)?

Анемічна модель добре підходить для примітивних додатків, де дані просто читаються і записуються. Проте домен управління тренажерним залом містить інваріанти та складні бізнес-правила: контроль перекриття розкладу занять (TimeRange), валідація тривалості абонементів (DateRange), перевірка статусів оплат (Invoice).

Використання Rich Domain Model дозволило **інкапсулювати** ці правила всередині самих сутностей. Замість того, щоб сервіс перевіряв, чи коректні дані, сутності самі генерують специфічні помилки (DomainError, MembershipErrors, ClassErrors). Сутності більше не є "тупими" контейнерами для даних — вони самостійно гарантують свою валідність. Це унеможливорює створення об'єкта в некоректному стані та робить поведінку системи передбачуваною і безпечною.